

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: GEOVANNY FROILAN ZAMBRANO MANZABA

PARALELO: A

SEMESTRE: 5TO

ACTIVIDAD: 1

PERIODO SEPTIEMBRE 2025 - ENERO 2026



Actividades a desarrollar en la Unidad 1

Actividad # 1

(Desarrollo de la actividad propuesta)

Actividad # 1: Estudio de caso - Sistema de Gestión de Incidentes (Help Desk)

Objetivo de la tarea:

Diseñar la arquitectura lógica de un sistema de gestión de incidentes técnicos (Help Desk) para infraestructura de servidores que permita aplicar correctamente patrones de diseño de software y flujos de trabajo profesionales (Gitflow), fortaleciendo la capacidad de análisis, abstracción y buenas prácticas en el desarrollo de soluciones escalables, reutilizables y mantenibles.

Descripción de la Actividad

El estudiante deberá diseñar un sistema de Help Desk orientado a la gestión de fallos en un Data Center. El sistema debe permitir reportar incidentes (hardware, red, software), asignar técnicos especializados y gestionar estados de resolución. Para lograr una arquitectura clara y profesional, se deberá utilizar obligatoriamente un repositorio en GitHub bajo el modelo Gitflow y aplicar al menos tres patrones de diseño de software, justificando su elección y uso técnico.

Análisis del problema

Identificar los actores principales

En el sistema de gestión de incidentes Help Desk intervienen tres actores principales que cumplen diferentes funciones dentro del proceso de soporte técnico.

1. **Usuario final:** El primer actor es el usuario final el cual representa las personas que utilizan los servicios tecnológicos del Data Center y reportan problemas relacionados con hardware, software y red.
2. **Técnico de soporte:** El segundo actor es el técnico de soporte el cual es el encargado de analizar, atender y resolver los incidentes registrados en el sistema.
3. **Administrador de sistemas:** Este es responsable de supervisar el funcionamiento general de la plataforma Help Desk, este nombramiento puede administrar los técnicos, controlar la asignación de incidentes, categorizar los fallos y supervisar el flujo de trabajo del sistema.



Definir los requerimientos funcionales (Crear ticket, categorizar fallo, asignar técnico) y no funcionales (Escalabilidad, mantenibilidad, trazabilidad de versiones)

Requerimientos funcionales

Los requerimientos funcionales para este sistema son los siguientes:

- Registrar tickets de incidentes técnicos
- Asignar técnicos especializados según el tipo de fallo
- Permitir consultas del estado de cada incidente
- Enviar notificaciones automáticas a usuarios y técnicos

Requerimientos no funcionales

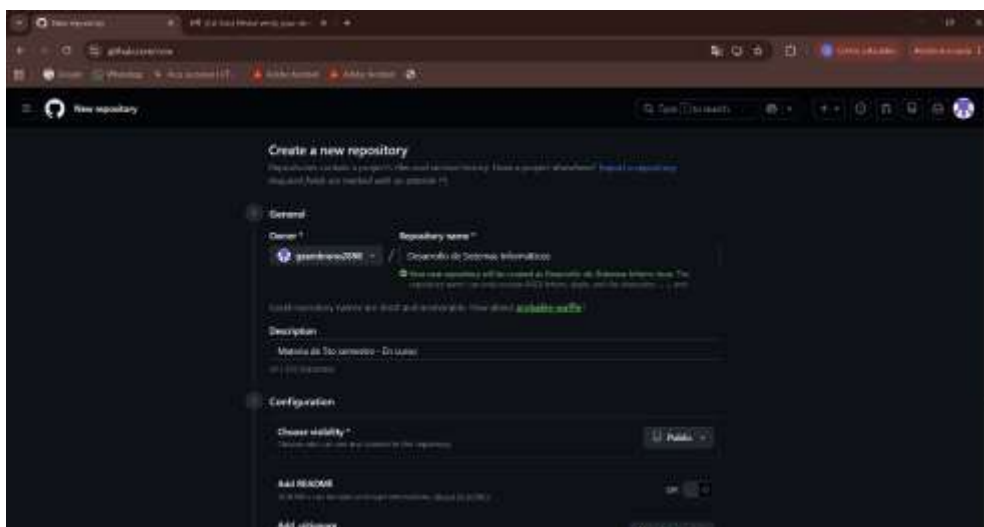
Los requerimientos no funcionales para este sistema son los siguientes:

- Garantizar escalabilidad del sistema
- Facilitar el mantenimiento y futuras actualizaciones
- Mantener seguridad y control de acceso de usuarios

Diseño del sistema

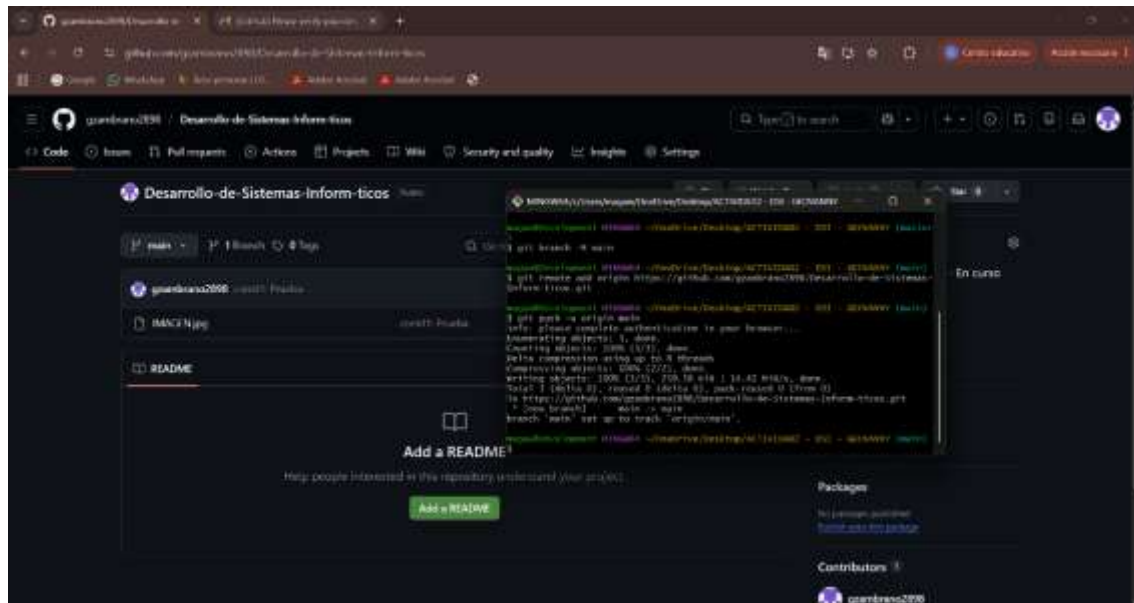
Crear un repositorio público en GitHub configurando la estructura de ramas Gitflow (master, develop y ramas feature/ para avances).

Para llevar a cabo este procedimiento, lo primero que se debe de realizar es un repositorio en github

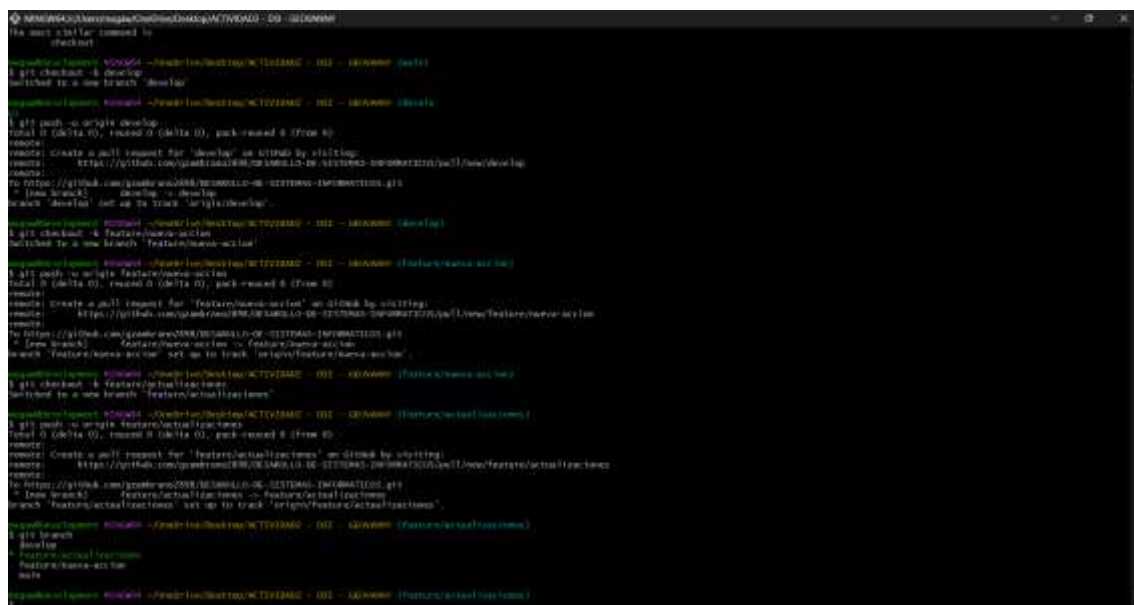




Una vez creado el repositorio podemos realizar una prueba subiendo un archivo de ejemplo

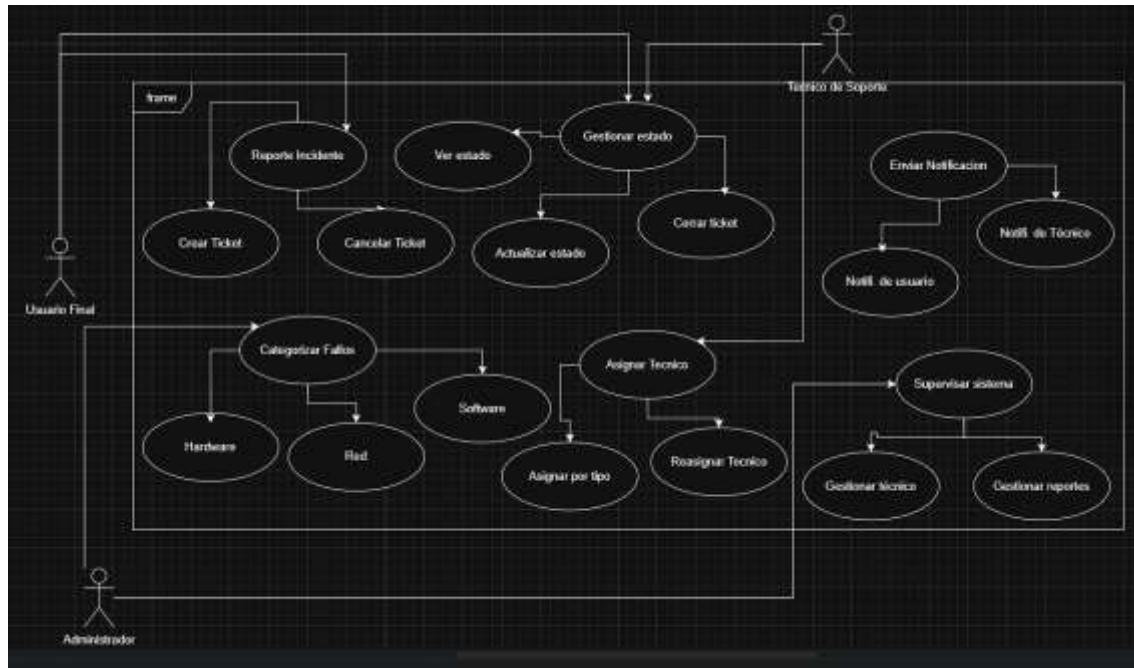


Hecha la comprobación aplicamos el funcionamiento por el flujo de versiones de GitFlow, vamos a crear ramas develop y feature

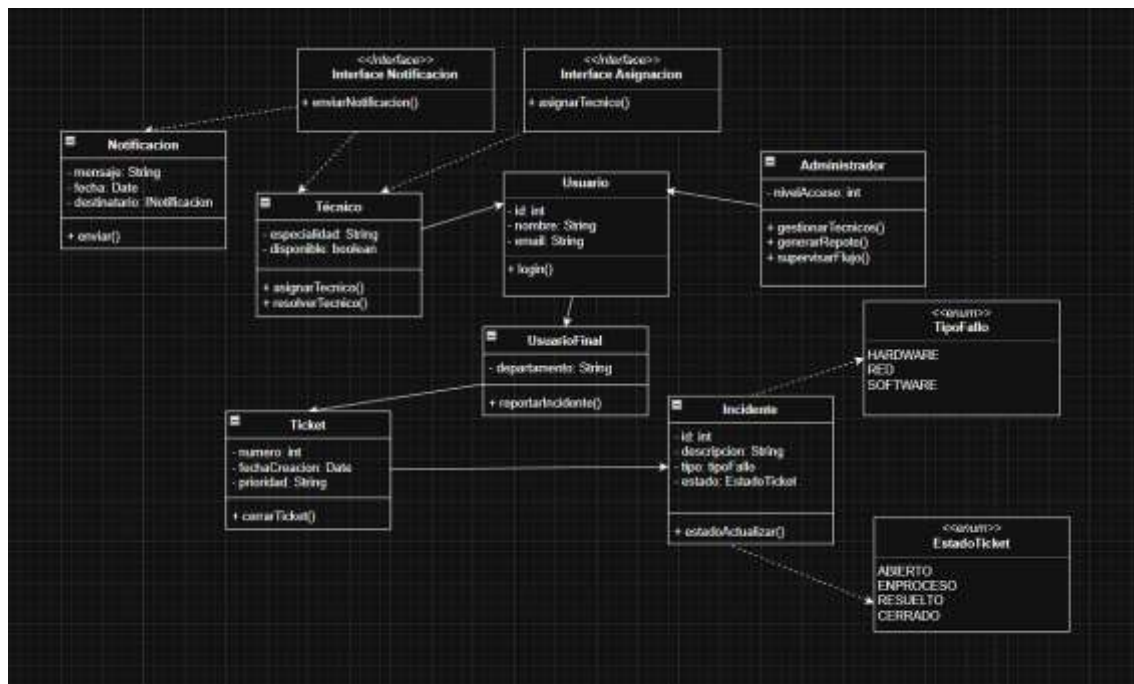


En este paso hemos creado las ramas y a su vez realizado un push para enviarla al repositorio de github

Crear un diagrama de casos de uso para entender las interacciones del personal de TI con la infraestructura.



Realizar un diagrama de clases UML donde se evidencien los objetos y su relación bajo principios SOLID.



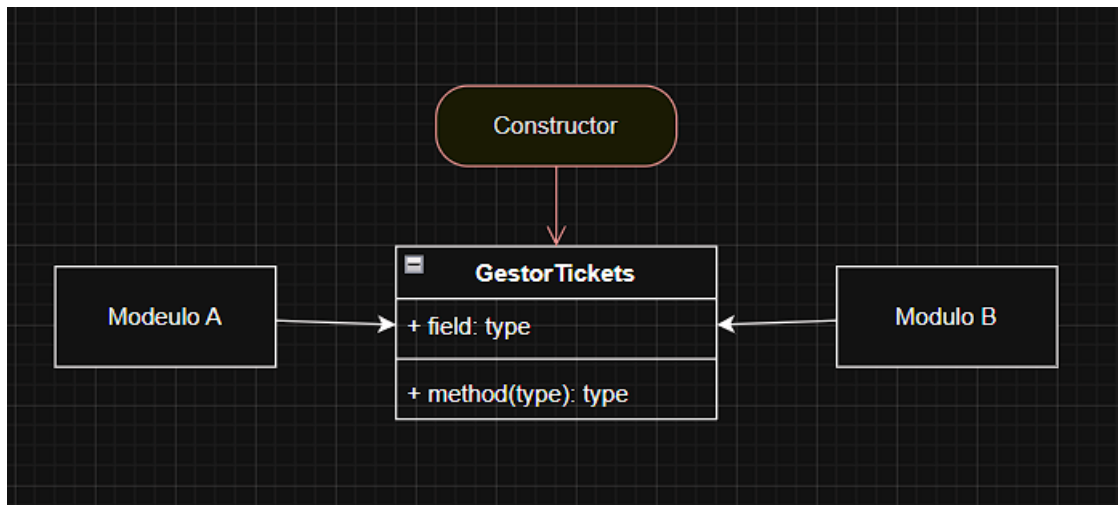


Diseñar la arquitectura general del sistema integrando el flujo de versiones

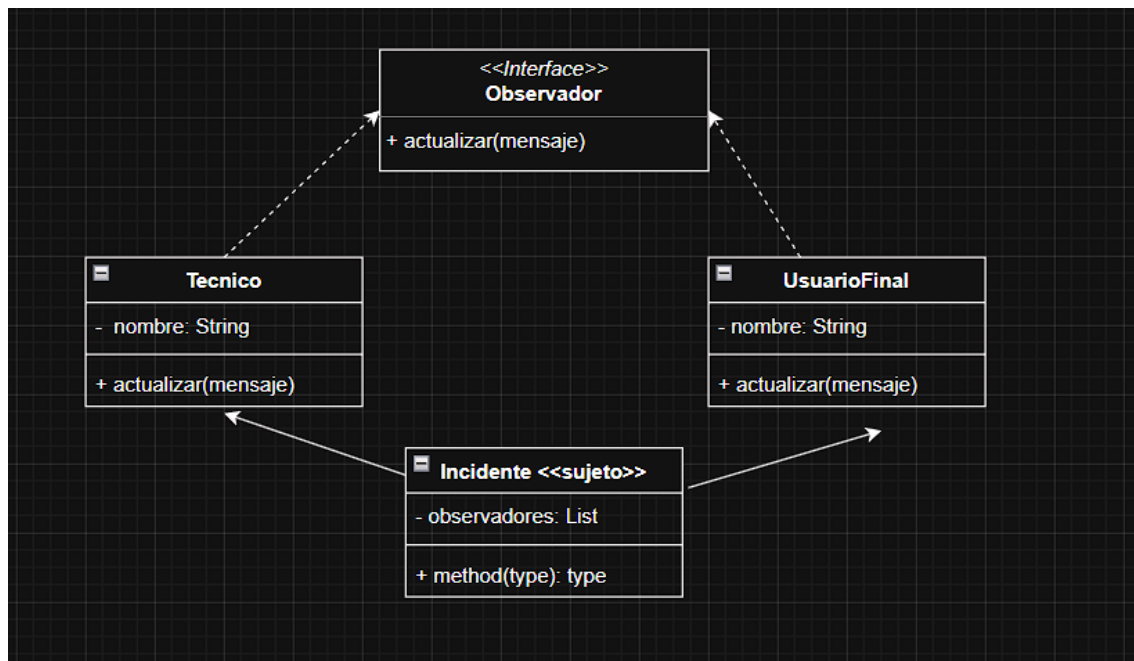
El Help Desk para el Data Center fue diseñado con una arquitectura en capas simple, donde cada componente del sistema tiene un papel específico. La capa de presentación es la que utiliza el usuario, ya sea el formulario para reportar un incidente o la interfaz para consultar el estado de su ticket. La capa de lógica de negocio constituye el núcleo del sistema, en ella residen las clases diseñadas como Incidente, Ticket, Técnico y Notificación, que se encargan de procesar todas las actividades. La capa de datos es responsable de almacenar y recuperar la información de la base de datos. Para manejar las versiones se utilizó el modelo Gitflow en GitHub. La rama master siempre refleja una versión estable y lista para ser entregada, y nadie trabaja directamente en ella. La rama develop es el lugar donde el equipo reúne los avances del sistema a medida que se van completando. Cada vez que se inicia un nuevo desarrollo, como el módulo de tickets o el sistema de notificaciones, se crea una rama feature a partir de develop, se trabaja en ella sin afectar a otros, y cuando está finalizada se fusiona de nuevo a develop a través de un pull request. Una vez que develop ha sido completamente probado y está libre de errores, ese código se integra a master como una versión oficial del sistema, como por ejemplo v1.0.

Aplicación de patrones de diseño

Aplicación de Patrón Singleton



Aplicación de diseño Observer



Documentación del diseño

Explicar el propósito de cada patrón utilizado y cómo resuelve un problema específico del Help Desk.

Explicacion del Patrón Singleton



El patrón Singleton se emplea para asegurar que solo exista una única instancia del gestor principal en el sistema Help Desk. Su objetivo es centralizar el manejo de tickets y la conexión a los datos, evitando la generación de múltiples instancias que podrían provocar inconsistencias. En el sistema, este patrón aborda el problema de mantener un control único y ordenado sobre la gestión de incidentes.

Explicacion del Patrón Observer

El patrón Observer facilita la implementación de un sistema de notificaciones automáticas entre los componentes del Help Desk. Su objetivo es notificar de manera instantánea a técnicos y usuarios cuando se produce un cambio en un ticket, como la creación de un incidente o la modificación de su estado. Este patrón soluciona el problema de la comunicación en tiempo real dentro del sistema, lo que mejora la rapidez y eficiencia del soporte técnico.

Justificar el uso de Gitflow: Incluir el enlace al repositorio de GitHub y capturas de pantalla del historial de ramas/commits que evidencien el flujo de trabajo.

Enlace de repositorio: <https://github.com/gzambrano2898/DESAROLLO-DE-SISTEMAS-INFORMATICOS.git>

A continuación, se presentan capturas donde justifican los commit que se han realizado y las ramas como evidencia que se aplicó el flujo de versiones, se visualiza la creación de una nueva rama la cual se llama diagramas-UML, donde se ha ubicado los diagramas que se han pedido

Se muestran las ramas creadas con el comando git branch y también en el repositorio de Github